

Pablo García Pérez
Mathematician | Data Scientist |
Python | AI

¿HASTA DÓNDE PODEMOS OPTIMIZAR UN ALGORITMO?

PYTHON & RUST


R

TOOLS

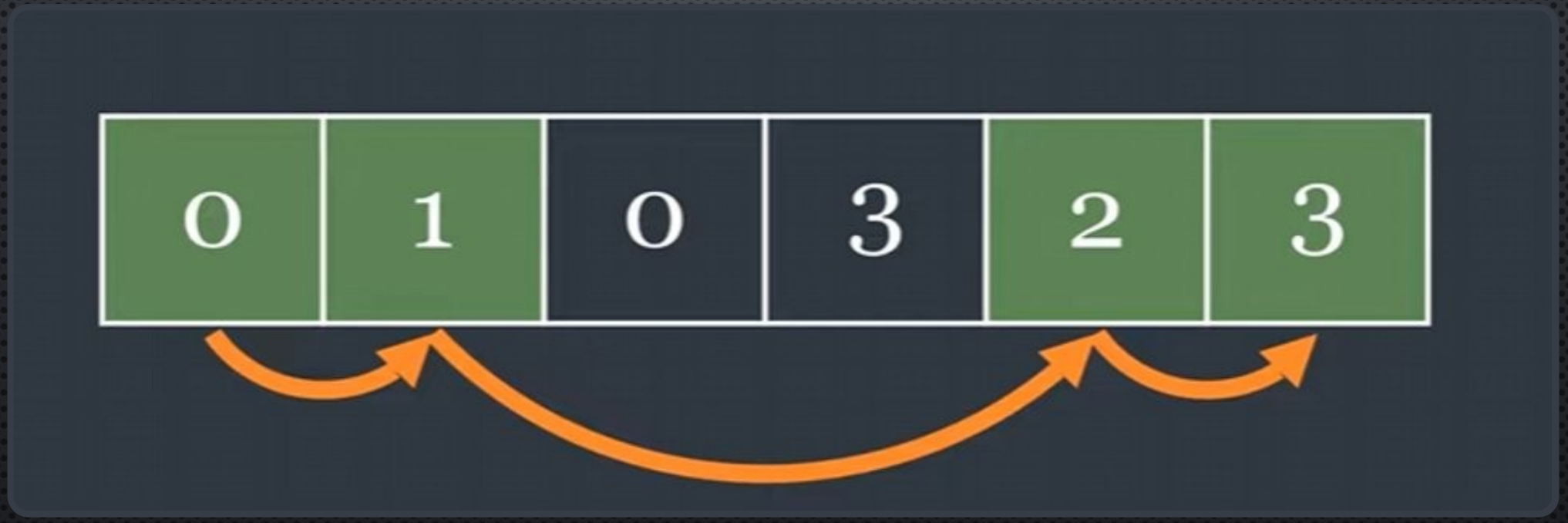
- PYTHON -> UV
- RUST -> RUST-SCRIPT
- BENCHMARKS -> HYPERFINE
- ANALYSIS -> MARIMO



LONGEST INCREASING SUBSEQUENCE



LONGEST INCREASING SUBSEQUENCE



```python
def lis_recursive(
    sequence: list[int],
    previous: float = float("-inf"),
    index: int = 0,
) -> list[int]:
    """ ... """
    # Base case: end of sequence
    if index == len(sequence):
        return []

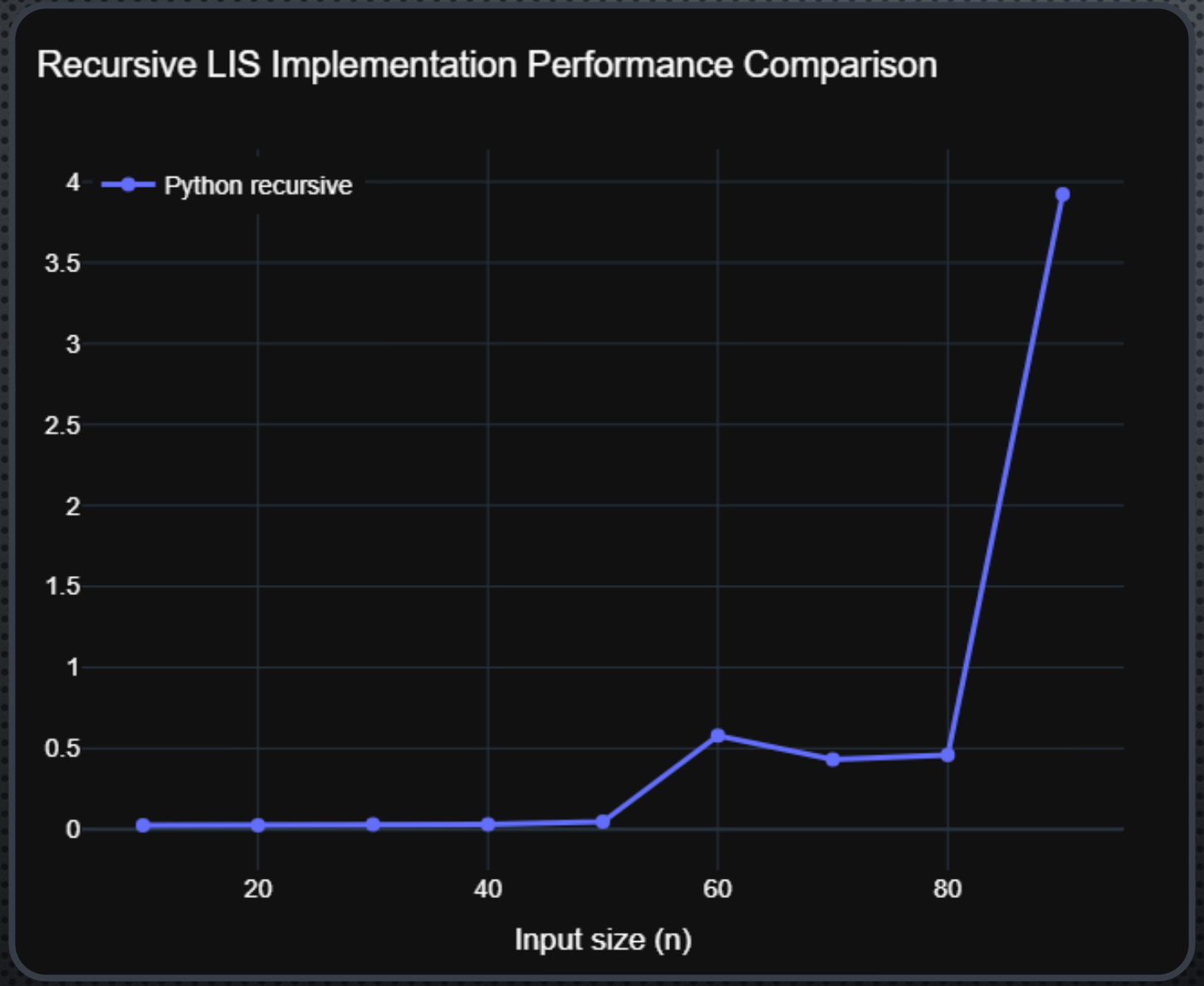
    # Option 1: skip current element
    subseq_excluding = lis_recursive(sequence, previous, index + 1)

    # Option 2: include current element if it's greater than the previous one
    subseq_including = []
    if sequence[index] > previous:
        subseq_including = [sequence[index]] + lis_recursive(
            sequence,
            sequence[index],
            index + 1,
        )

    # Return the longer subsequence
    return (
        subseq_including
        if len(subseq_including) > len(subseq_excluding)
        else subseq_excluding
    )
```

PYTHON RECURSIVE IMPLEMENTATION

O(2^N)



RECURSIVE IMPLEMENTATION

O(2^N)

```rust
pub fn lis_recursive(sequence: &[i32]) -> Vec<i32> {
    fn helper(sequence: &[i32], prev: i32, index: usize) -> Vec<i32> {
        if index == sequence.len() {
            return Vec::new();
        }

        // Option 1: skip current element
        let exclude = helper(sequence, prev, index + 1);

        // Option 2: include current element if it's greater than previous
        if sequence[index] > prev {
            let mut include = helper(sequence, sequence[index], index + 1);
            include.insert(0, sequence[index]); // prepend current element

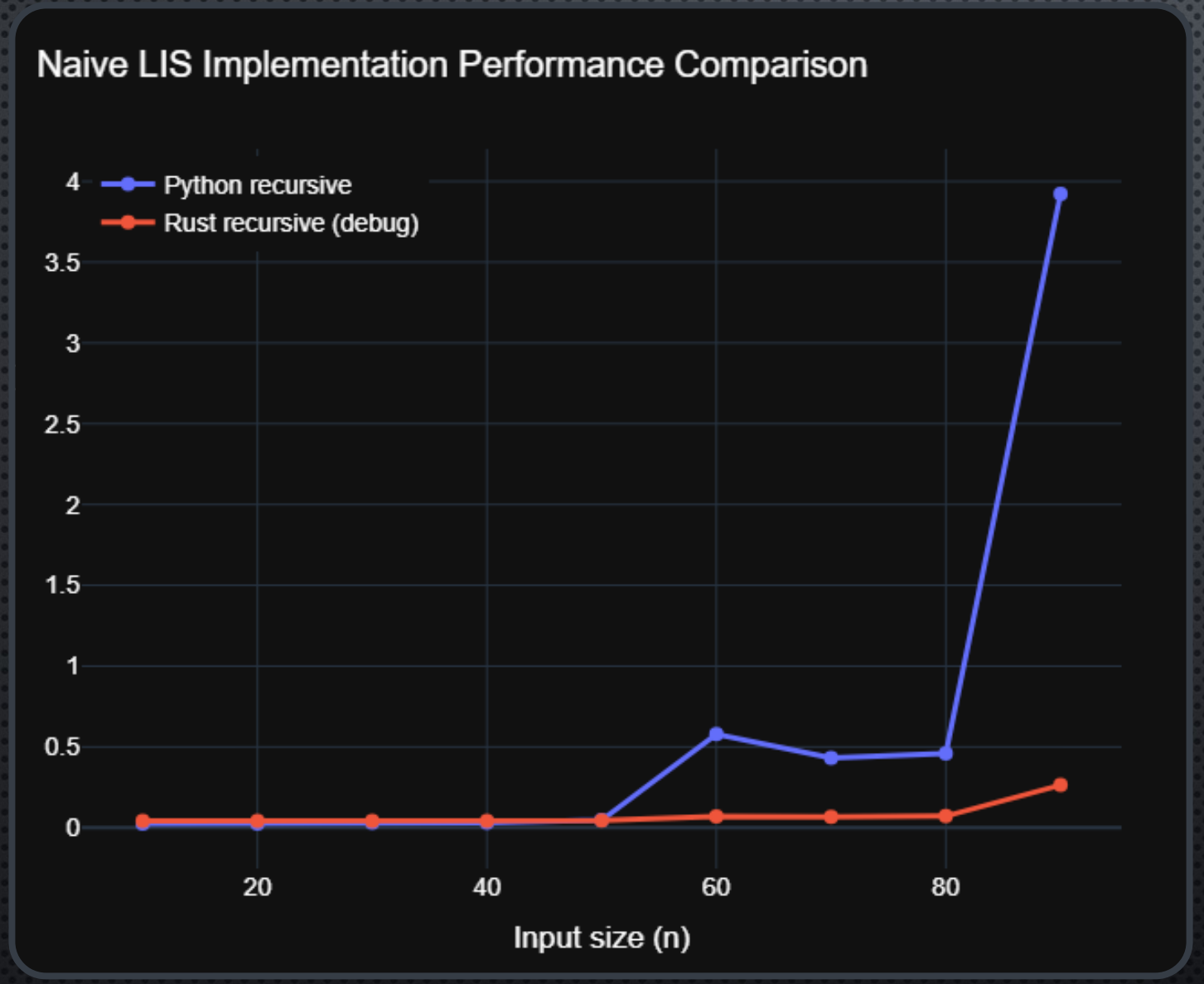
            // Return the longer subsequence
            if include.len() > exclude.len() {
                return include;
            }
        }

        exclude
    }

    helper(sequence, i32::MIN, 0)
}
```

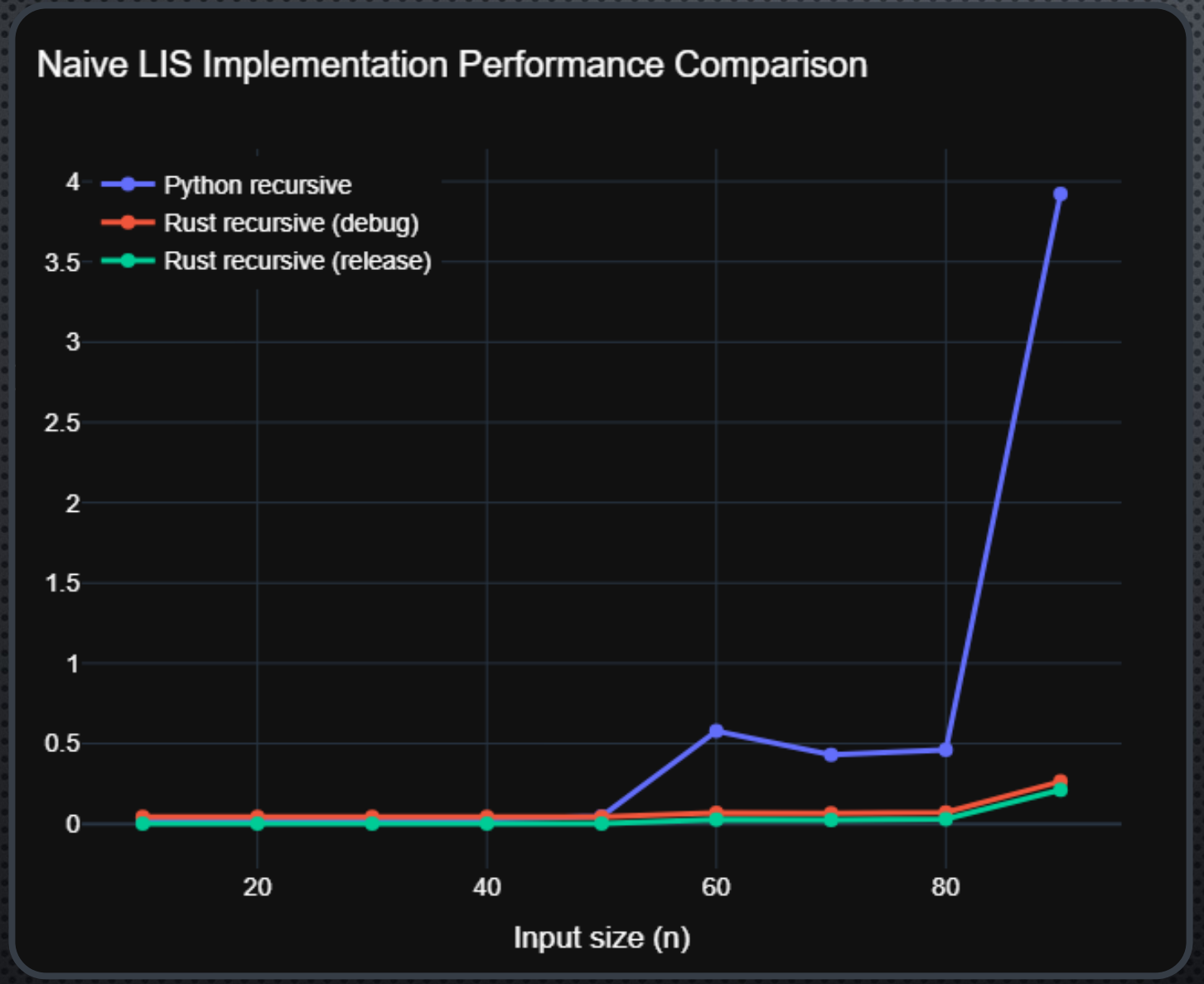
RUST RECURSIVE IMPLEMENTATION

O(2^N)



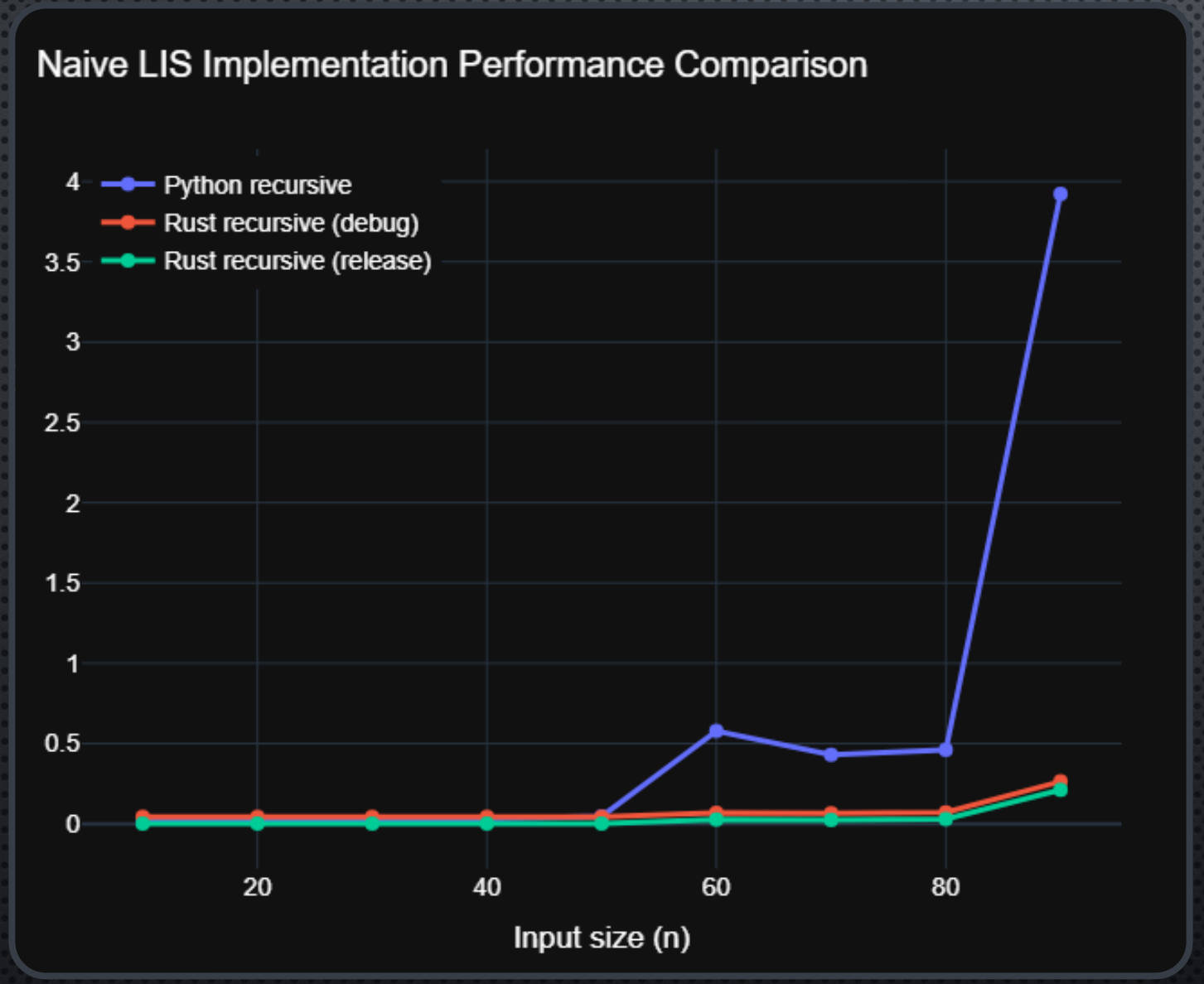
RECURSIVE IMPLEMENTATION

O(2^N)



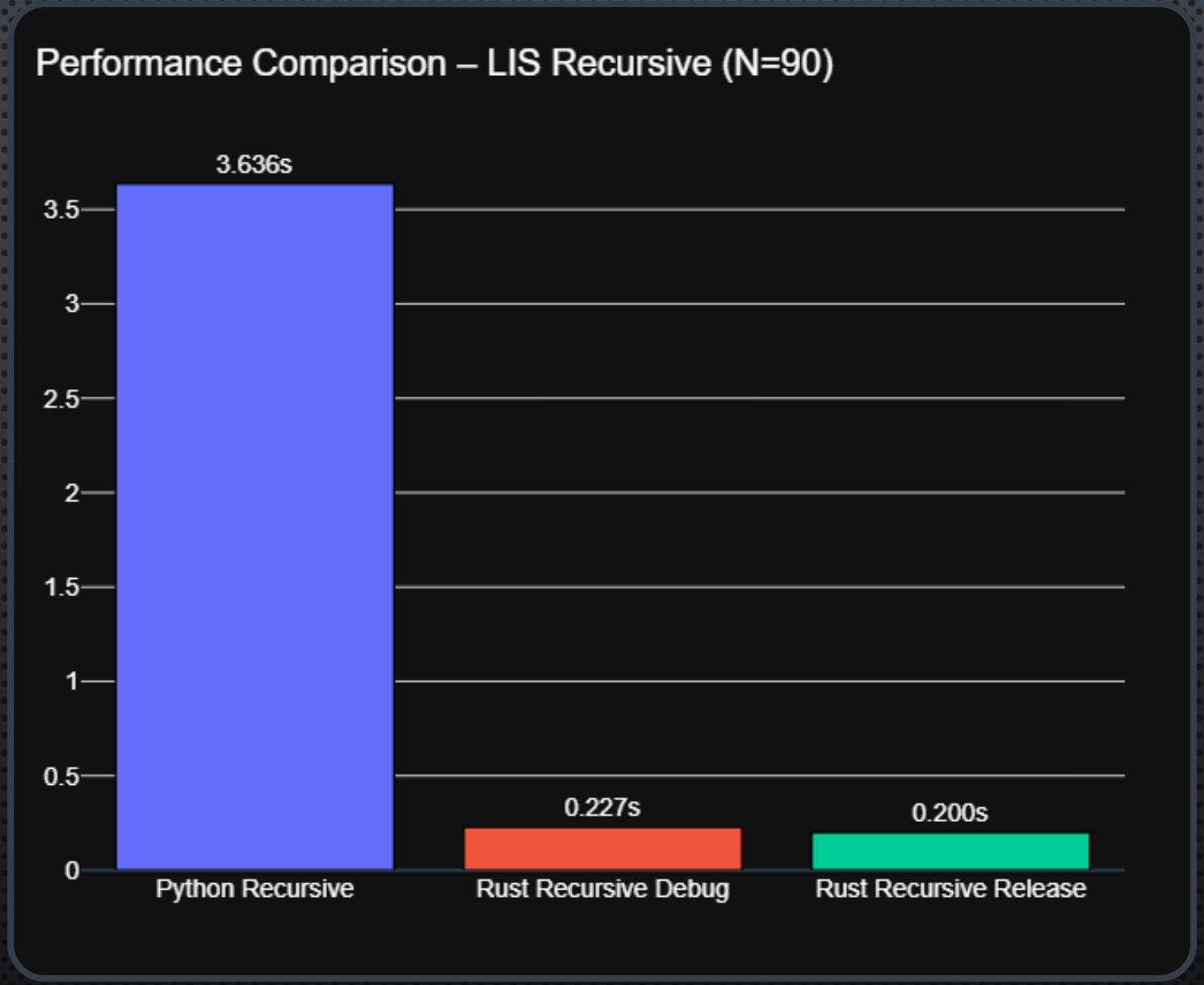
RECURSIVE IMPLEMENTATION

O(2^N)



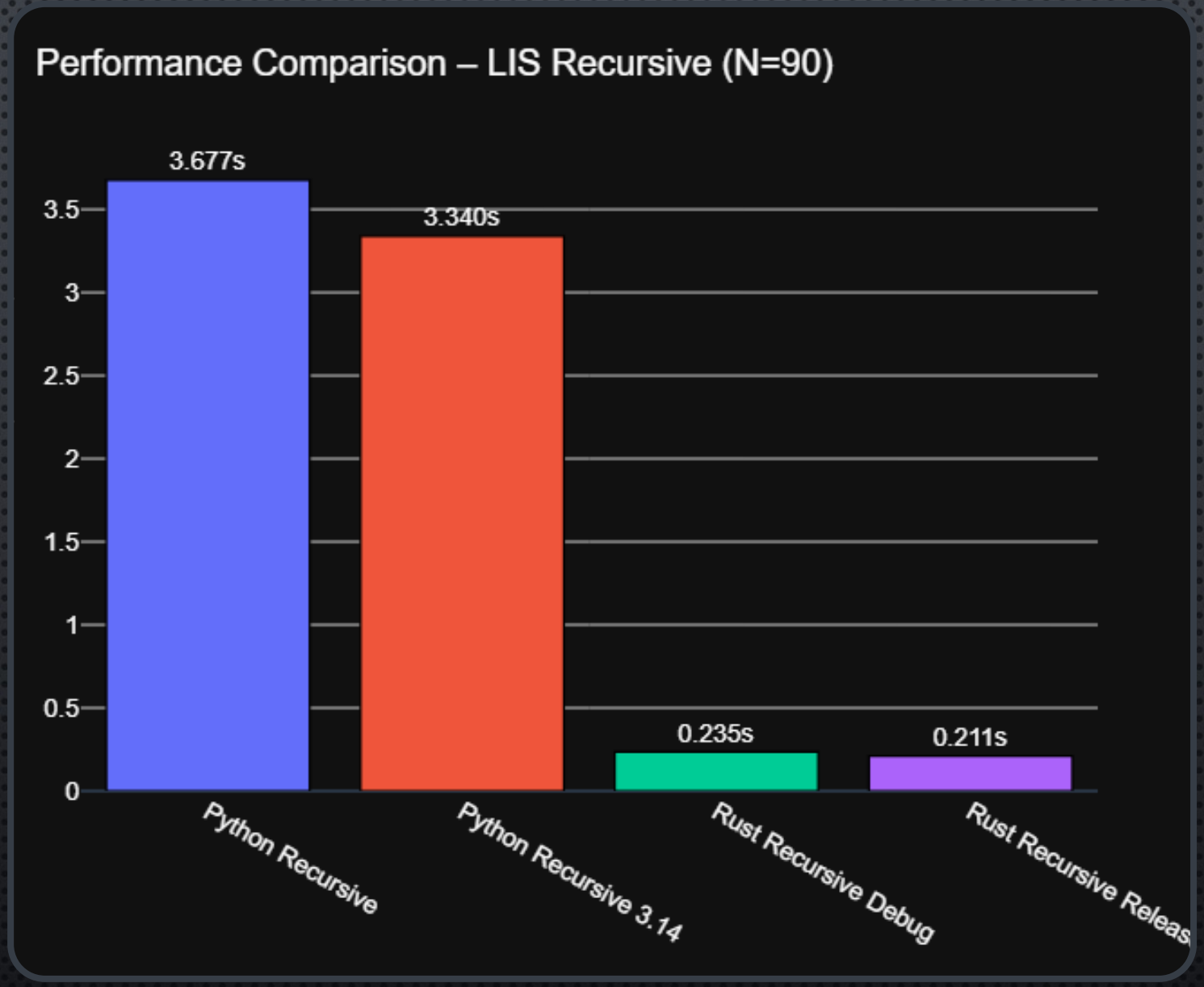
RECURSIVE IMPLEMENTATION

O(2^N)

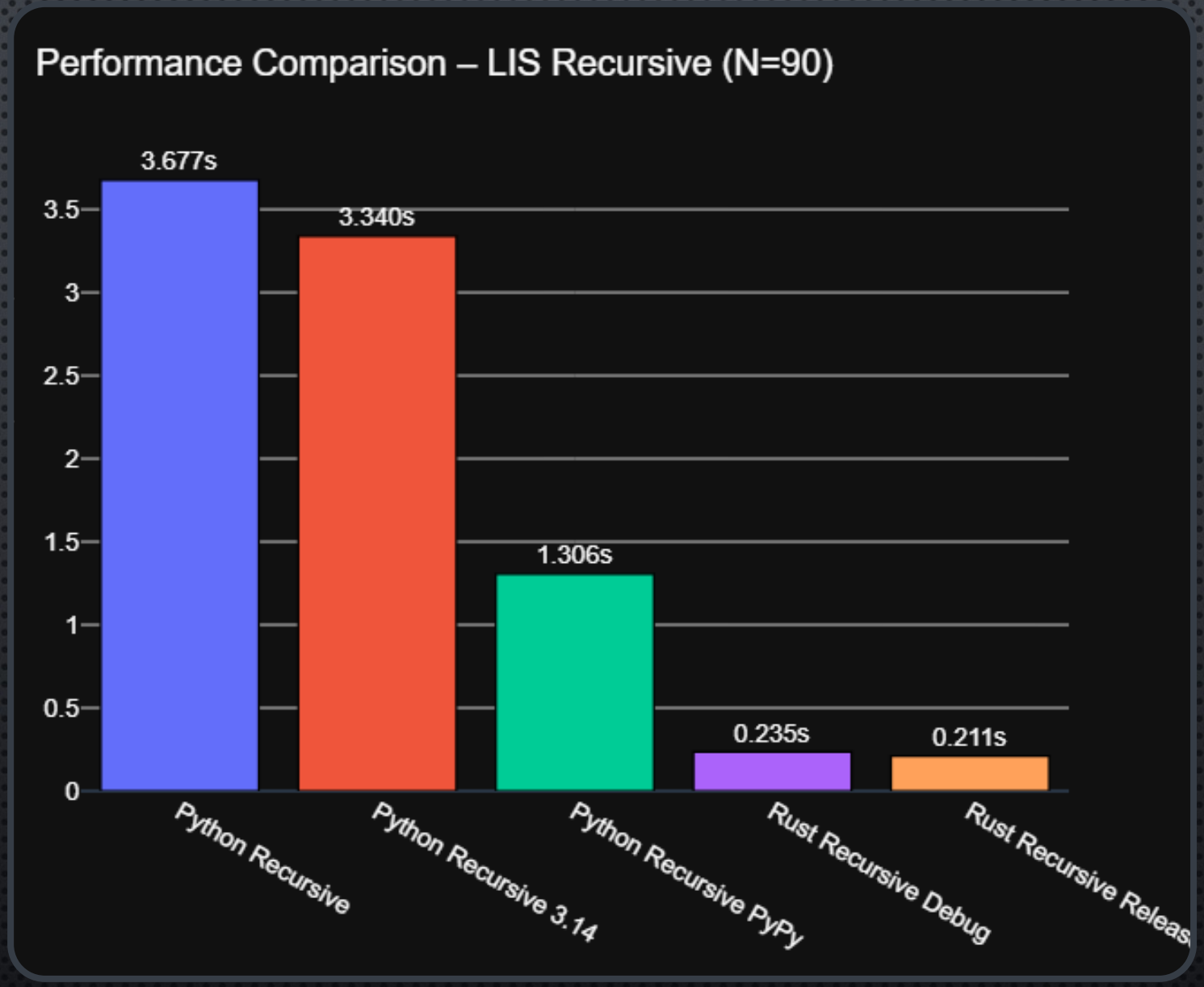

Performance Comparison – LIS Recursive (N=90)
3.636s
0.227s
0.200s
3.5
3
2.5
2
1.5
1
0.5
0
Python Recursive
Rust Recursive Debug
Rust Recursive Release

O(2^N) -> O(N^2) -> O(N LOG(N)) ???


PYTHON
3.14


Performance Comparison – LIS Recursive (N=90)
3.677s
3.340s
0.235s
0.211s
3.5
3
2.5
2
1.5
1
0.5
0
Python Recursive
Python Recursive 3.14
Rust Recursive Debug
Rust Recursive Releas


pypy


Performance Comparison – LIS Recursive (N=90)
3.677s
3.340s
1.306s
0.235s
0.211s
3.5
3
2.5
2
1.5
1
0.5
0
Python Recursive
Python Recursive 3.14
Python Recursive PyPy
Rust Recursive Debug
Rust Recursive Releas

```python
def lis_recursive_opt(
    sequence: list[int],
    previous: float = float("-inf"),
    index: int = 0,
) -> list[int]:
    """ ... """
    # Base case: end of sequence
    if index == len(sequence):
        return []

    # Option 1: skip current element
    subseq_excluding = lis_recursive_opt(sequence, previous, index + 1)

    # Compute the current element once
    current = sequence[index]

    # Option 2: include current element if it's greater than the previous one
    subseq_including = []
    if current > previous:
        subseq_including = [current] + lis_recursive_opt(
            sequence,
            current,
            index + 1,
        )

    # Return the longer subsequence
    return (
        subseq_including
        if len(subseq_including) > len(subseq_excluding)
        else subseq_excluding
    )
```

PYTHON BRUTE FORCE IMPLEMENTATION

O(2^N)

```python
def lis_recursive(
    sequence: list[int],
    previous: float = float("-inf"),
    index: int = 0,
) -> list[int]:
    """ ... """
    # Base case: end of sequence
    if index == len(sequence):
        return []

    # Option 1: skip current element
    subseq_excluding = lis_recursive(sequence, previous, index + 1)

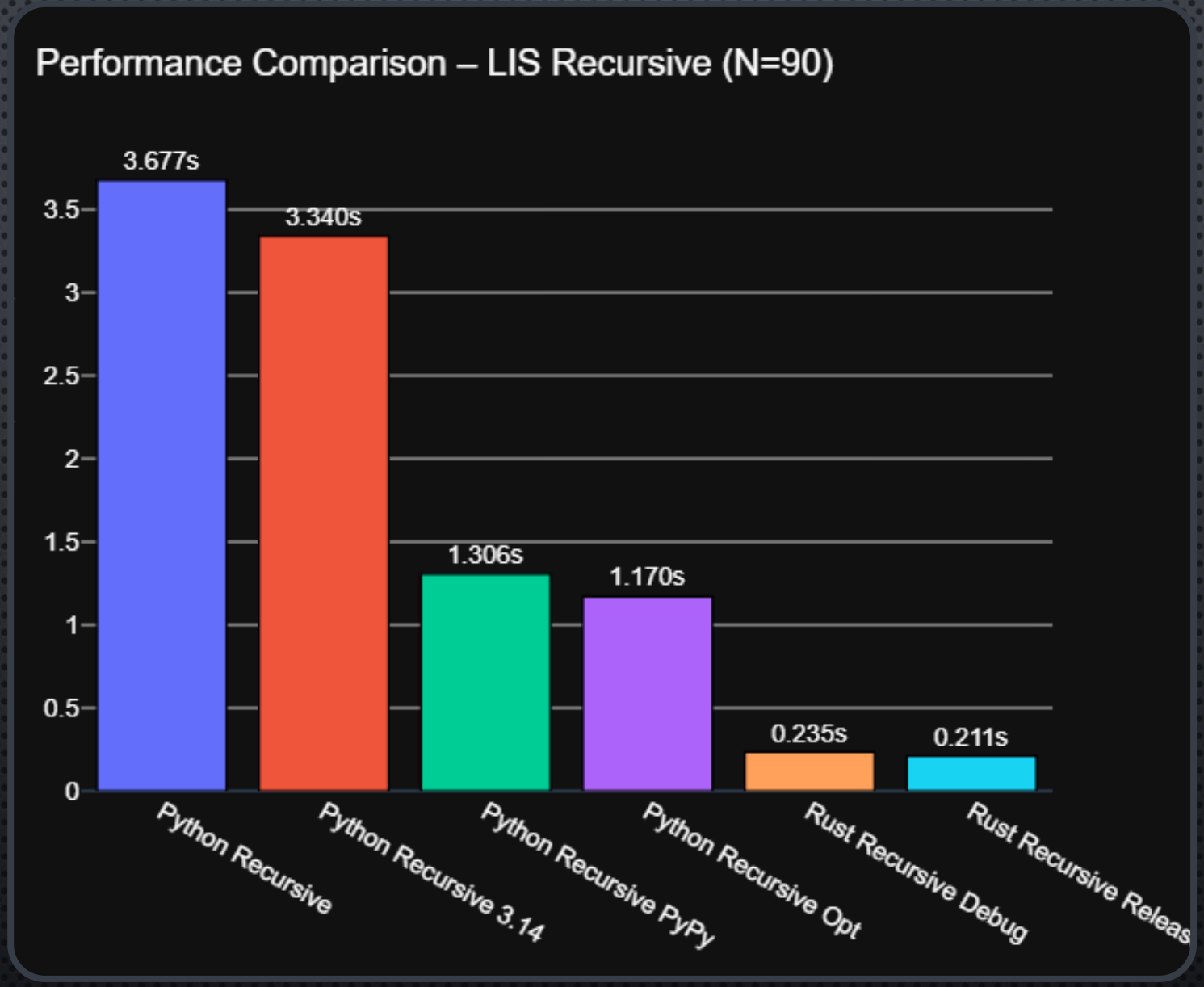
    # Compute the current element once
    current = sequence[index]

    # Option 2: include current element if it's greater than the previous one
    if current > previous:
        subseq_including = [current] + lis_recursive(
            sequence,
            current,
            index + 1,
        )
        if len(subseq_including) > len(subseq_excluding):
            return subseq_including

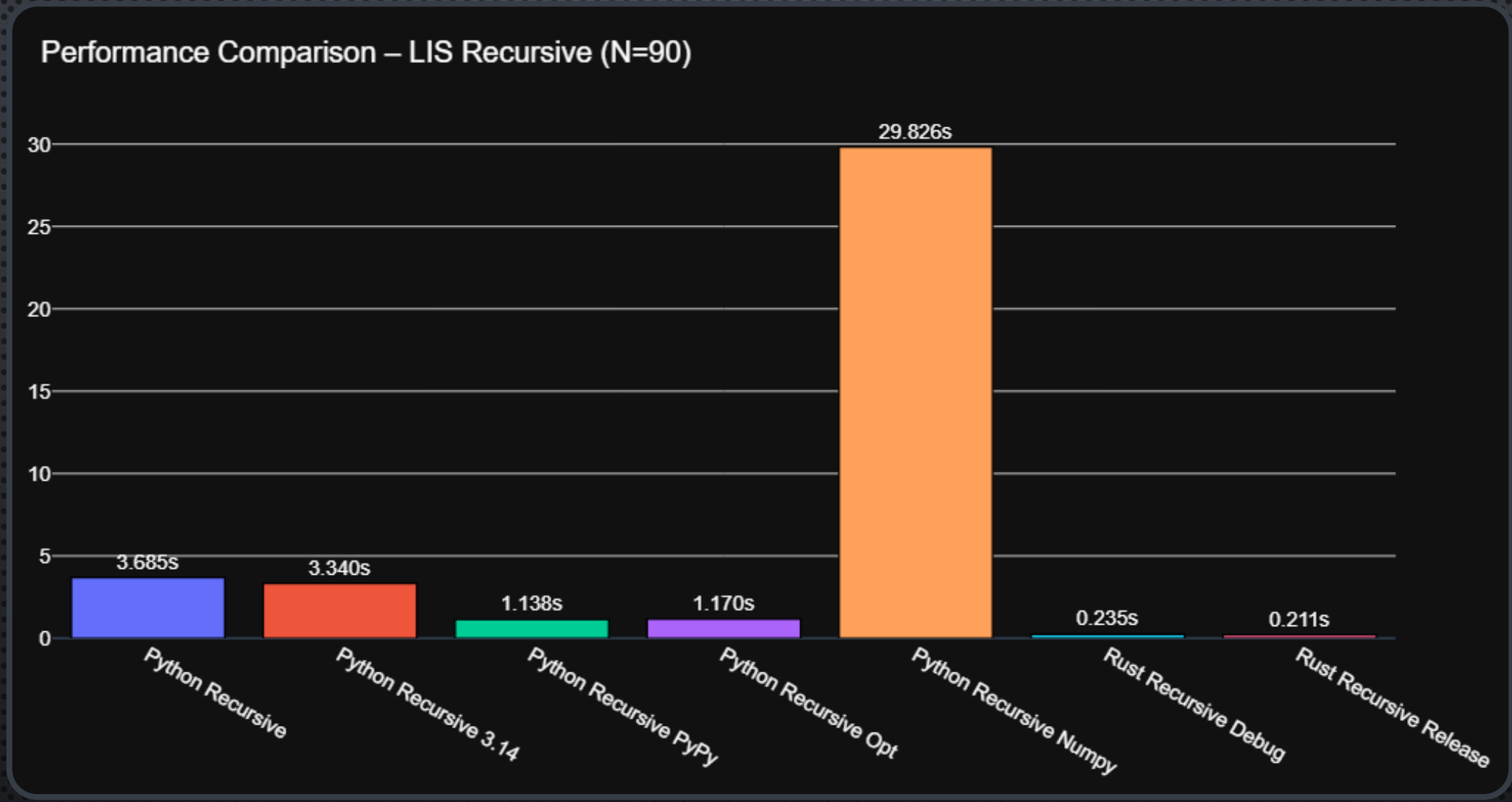
    return subseq_excluding
```

PYTHON BRUTE FORCE IMPLEMENTATION

O(2^N)


Performance Comparison – LIS Recursive (N=90)
3.677s
3.340s
1.306s
1.170s
0.235s
0.211s
3.5
3
2.5
2
1.5
1
0.5
0
Python Recursive
Python Recursive 3.14
Python Recursive PyPy
Python Recursive Opt
Rust Recursive Debug
Rust Recursive Releas


NumPy


Performance Comparison – LIS Recursive (N=90)
30
25
20
15
10
5
0
3.685s
3.340s
1.138s
1.170s
29.826s
0.235s
0.211s
Python Recursive
Python Recursive 3.14
Python Recursive PyPy
Python Recursive Opt
Python Recursive Numpy
Rust Recursive Debug
Rust Recursive Release

FURTHER OPTIMIZATIONS



Pablo García Pérez
pablofueros · he/him

¿HASTA DÓNDE PODEMOS OPTIMIZAR UN ALGORITMO?

PYTHON & RUST